

SLE-3: Architectural Design (Full C4 Model)

Course: 02AML204 – Introduction to Artificial Intelligence

PRN No : 26UAM302

Name : Jyoti Prakash Chavan

Division : A

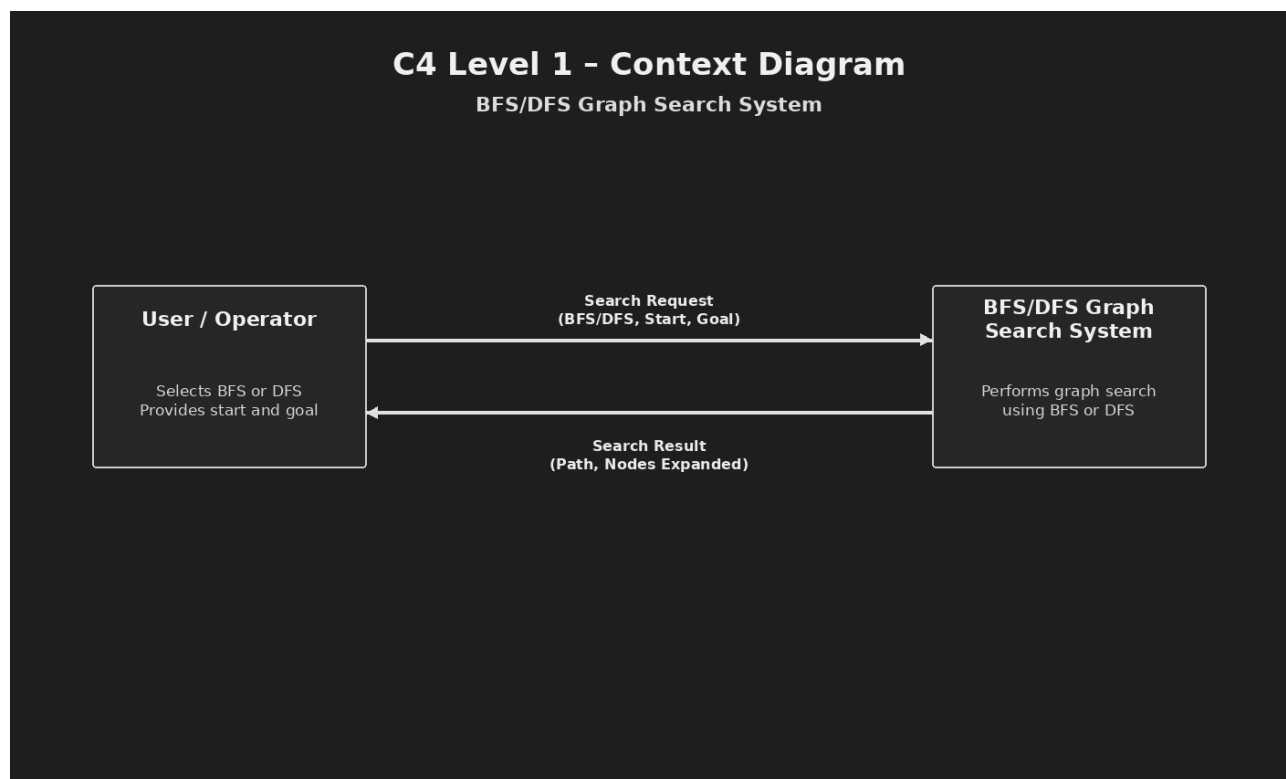
Date : 04/09/2026

GitHub link : <https://github.com/Jyotichavan07/IAI-SLE3-26UAM302>

1. System Title & Short Description

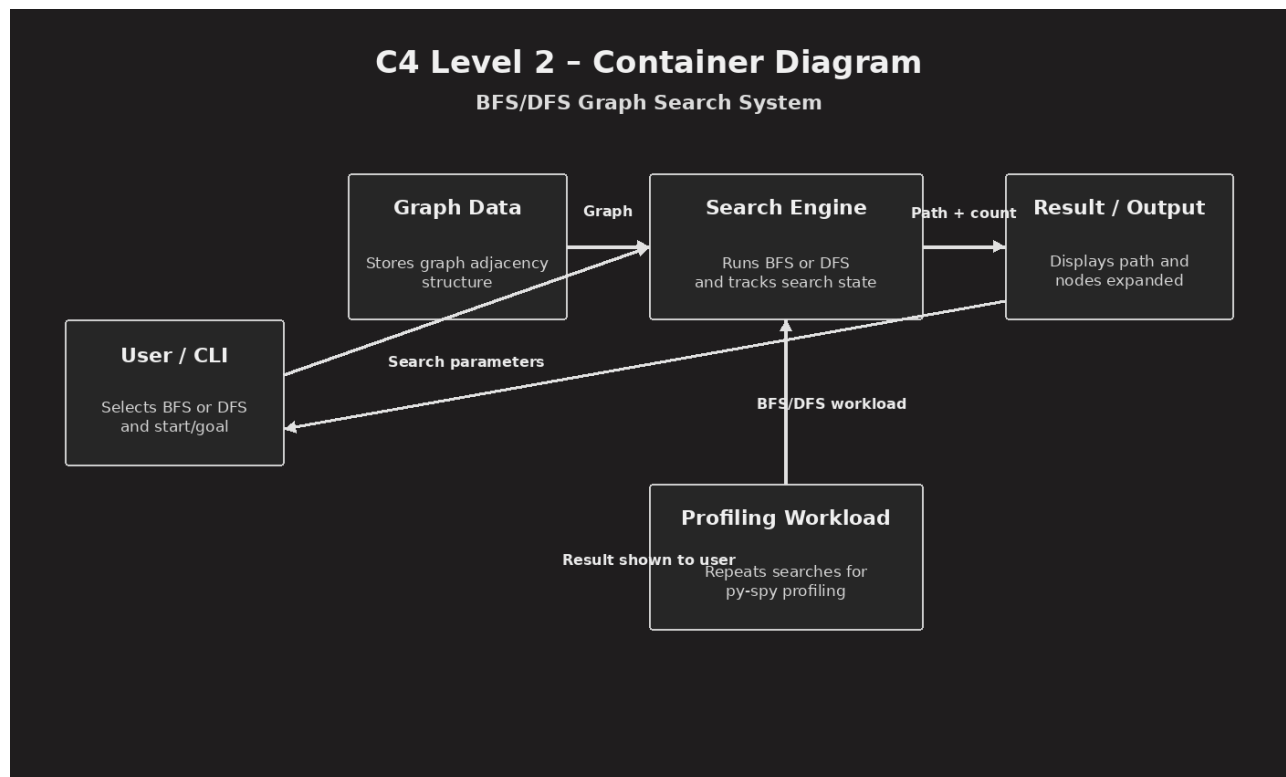
The project is a simple Graph Search System that implements two search algorithms: Breadth-First Search (BFS) and Depth-First Search (DFS). The program uses a predefined graph and searches from a start node to a goal node. It displays the path found and the number of nodes expanded. In SLE-2, the search functions are also repeated many times so their execution can be profiled using py-spy.

2. Context Diagram — Level 1



The user selects BFS or DFS and gives the start and goal nodes. The Graph Search System then performs the selected search and gives the result back to the user. The result mainly contains the path found and the number of nodes expanded.

3. Container Diagram — Level 2

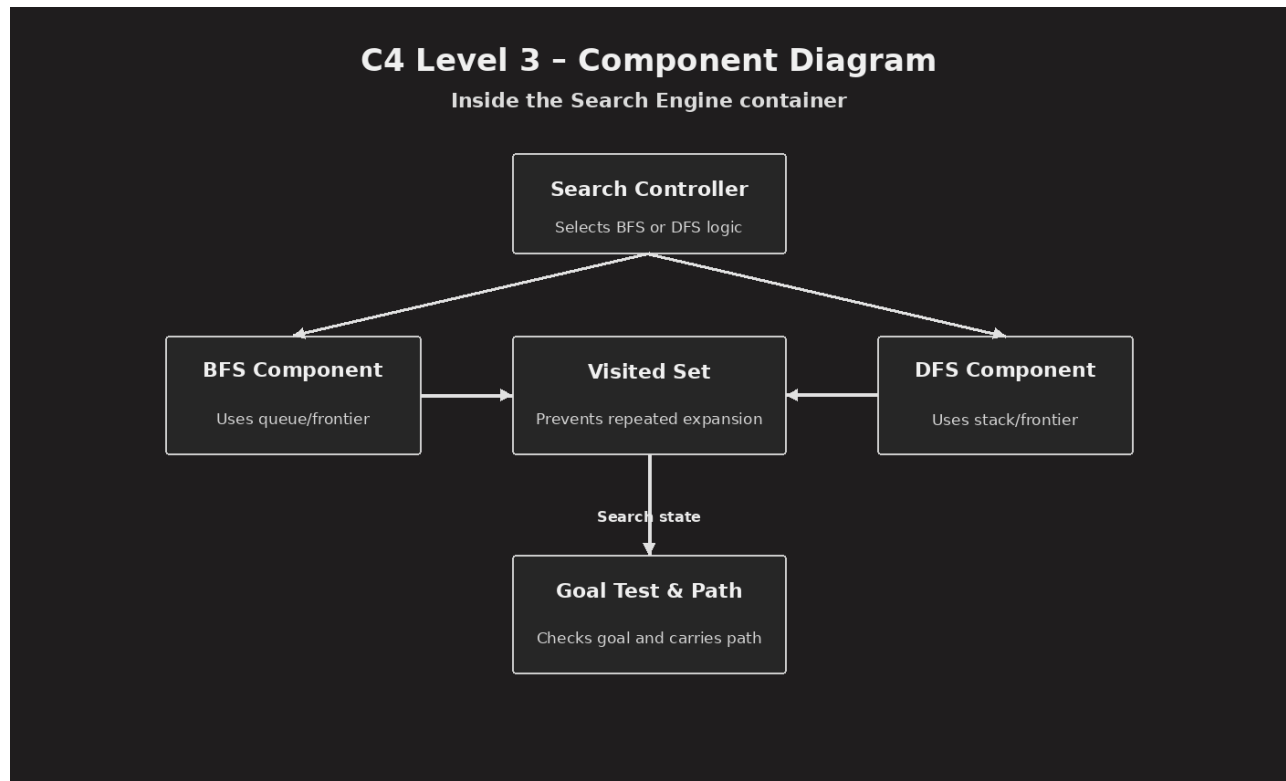


The User/CLI part takes the algorithm choice and search values. Graph Data stores the adjacency-list representation used by the program. The Search Engine contains the BFS and DFS logic. Result/Output prints the path and nodes expanded. The Profiling Workload repeats the search so that py-spy has enough execution activity to sample.

4. Component Diagram — Level 3

C4 Level 3 - Component Diagram

Inside the Search Engine container



The Search Engine is the main container, so its important internal parts are shown here. The Search Controller decides which algorithm is selected. BFS uses a queue, while DFS uses a stack. Both use a visited set to avoid expanding the same node again. The goal test checks whether the current node is the required goal, and the path information is carried with the search entries.

5. Code Level Overview — Level 4

- `graph` : stores the graph and the neighbours of each node.
- `bfs(graph, start, goal)` : searches the graph using a queue.
- `dfs(graph, start, goal)` : searches the graph using a stack.
- `run_bfs_workload()` : repeats BFS 100,000 times for profiling.
- `run_dfs_workload()` : repeats DFS 100,000 times for profiling.
- `display_results()` : prints the selected algorithm, path and nodes expanded.
- `argparse` : allows the user to choose bfs or dfs from the command line.

6. Design Decisions

I kept BFS and DFS as the main Search Engine because they are the main algorithms in my program. BFS uses a queue and DFS uses a stack, which makes their search behaviour different.

I kept the profiling workload separate from the search functions so that the same search code can be used for both the result and the SLE-2 py-spy experiment. The architecture is kept small because this is a simple graph-search program.

7. AI Contribution Note

- ✓ AI tool used : ChatGPT.
- ✓ I used AI to understand the C4 model, decide how the existing BFS/DFS program can be represented at the Context, Container, Component and Code levels, and to improve the wording and presentation of the report.
- ✓ The original BFS/DFS program, graph, start and goal settings, repetitions and py-spy workload were provided as my SLE-2 work.
- ✓ I will review the diagrams and explanations and make sure I can explain them during evaluation.

8. Conclusion

Through this SLE, I understood how the same program can be explained at different architecture levels. The Context diagram shows the user and the complete system, the Container diagram shows the main parts, and the Component diagram focuses on the Search Engine. The Code level connects these design parts with the actual Python functions. This helped me understand the structure of my BFS/DFS program instead of looking only at the source code.